VReAct: Verification as an Architectural Layer for LLM Agents

A V-Model Design, and a Pilot Study of Its Deterministic Core

Ali Afoud
Independent Researcher
Canada

Jie JW Wu
Michigan Technological University
Houghton, USA

arXiv:2609.31937v3 [cs.SE] 18 Oct 2026

Abstract

Large language model (LLM) agents built on the ReAct pattern concentrate four responsibilities in a single model: selecting a strategy, choosing each action, formatting it, and judging whether the result is adequate. No component outside the generative loop can reject the loop's output, so an agent that cannot make progress does not report failure; it continues until an externally imposed budget terminates it. We propose VReAct, where we treat verification as an architectural layer for ReAct agent systems by adapting the V-model from software engineering: specification levels descend from requirements to individual steps, each level is paired with a dedicated verifier, a deterministic controller enforces every verdict, and memory is written only by verification outcomes, so that a rejection localizes the level that introduced the fault and an agent halts by declining rather than by exhaustion. Each verifier separates a zero-cost deterministic *gate* from an optional LLM *judge*, allowing the contribution and cost of each to be measured independently. We report a pilot study implementing the acceptance-level and unit-level verifier pairs, comparing five configurations that share one executor, tool set, and scorer and differ only in their verification components, on the four-hop stratum of MuSiQue with an 8B-parameter backbone. Across 47 executions, the two unverified configurations answered none of ten questions, every run ending at a step cap or provider token limit; the verified configuration without a planner answered eight of ten and abstained on the rest. Deterministic gates produced eight of the nine observed corrections at zero marginal cost, and planning degraded performance once verification was present. These results characterize termination behavior rather than accuracy at scale; we outline a twelve-month plan for completing and evaluating the full architecture, including the integration-level pair the pilot omits.

1 Introduction

An LLM agent is a language model in a control loop: it receives a goal, invokes tools, and continues until it decides it has finished. The prevailing design is ReAct [1], which extends chain-of-thought prompting [2] to an interactive setting by alternating a *Thought*, which modifies only the model's context, with an *Action*, whose *Observation* is appended to that context. Most deployed frameworks are this loop with a tool registry attached [5, 6], and the pattern is now common in software engineering [36, 37]. The design concentrates four responsibilities in one model and one prompt: selecting the overall strategy, selecting each action, formatting that action, and judging whether the resulting observation is sufficient. No component outside the loop can reject the loop's output.

The practical problem is not primarily accuracy but the absence of a *mechanism for declining*. A ReAct loop that cannot make progress does not report failure; it continues to emit actions until an externally imposed budget intervenes or, absent a budget, until the serving infrastructure refuses further requests. Such an agent cannot be composed into a larger system, because a caller receives either a plausible answer or nothing and cannot distinguish the two. The literature documents the underlying mechanics: turn-by-turn action selection is a greedy policy that can be arbitrarily suboptimal over long horizons, with reported myopic-trap rates above 50% and post-error recovery near 5% [10]; agents abandon plans they produced, repeat executed steps, and act inconsistently with the preceding thought [22]; and token consumption grows quadratically because the full history is resubmitted each turn [7]. A larger budget does not help: the agent commits to an unproductive trajectory early [8].

Existing work addresses the symptoms rather than the structure. Planning and efficiency methods change what the agent reads or how far ahead it looks [7–10], and prompting strategies improve how it reasons [3, 4], but neither inspects the result. Where checking does exist, it is the agent checking itself [11, 12], which is the configuration that Huang et al. [14] report to fail and that Zheng et al. [15] report to be biased. Table 1 summarizes the four gaps that remain and how this work addresses each; the pilot closes G3 and makes partial progress on two others.

Our response is architectural: we argue that the missing structure is a verification layer, and that software engineering already provides its template. Software engineering separated construction from certification decades ago, and the V-model pairs each design stage with a test stage of matching scope. We transfer that structure to LLM agents. It is the architecture, rather than the pilot study, that this line of work ultimately seeks to establish.

1.1 What "verification" means here

The term *verification* is used loosely in the agent literature, covering self-critique, self-consistency, confidence estimation, reranking, and formal proof; in LLM training it now also names reward models scoring reasoning steps [16]. We use it in neither sense but in the software-engineering one [41], and we claim no formal guarantees: our checks decide predicates on one artifact of one execution, as in runtime verification [42], not properties over all executions. A *verification act* satisfies four conditions.

- *V1, externality*: the verdict comes from a component distinct from the one that produced the artifact: a restricted-input invocation, ordinary program logic, or, preferably, a separate model.
- *V2, authority*: the verdict is binding and enforced by the controller; it is not returned as advice the generator may ignore.
- *V3, pairing*: each verifier inspects artifacts at exactly one specification level, so the level at which a rejection fires

Table 1: The four gaps this work addresses.

	What the literature does	The gap	Our fix
G1	The model that produced the output also checks it [11, 12].	No agent has a checker *outside* the loop whose verdict it must obey. Self-checking without external feedback degrades accuracy [14, 15].	Verifiers are separate components. The controller enforces their verdicts, so the agent cannot proceed past a rejection.
G2	External checks exist, but for one artifact type: a reasoning step [16], a program [17].	None pairs checks to the levels of a specification, so a rejection cannot say *which* stage broke, and recovery means starting over.	One verifier per level. A rejection names the level that failed, and only that level is redone.
G3	Systems mix free structural checks with LLM judgement and report the combined effect.	Nobody has measured what each half contributes, so the layer cannot be priced.	Gates and judges are separate stages, and every correction is attributed to one of them.
G4	Context is trimmed to fit a budget [7, 8].	Nothing decides what deserves to be carried forward: verified and failed work are resubmitted alike.	Only verdicts write memory. Accepted results go to a ledger, rejections to an index keyed by component and cause.

identifies where the fault was introduced and scopes the retry.

- *V4, bounded termination*: each level's rejection budget is finite, and exhausting it yields an abstention naming the level that failed, not a degraded answer.

Three terms recur. A *gate* is a deterministic verifier written as program logic—a regular expression, a set-membership test, or, in code, a compiler or test runner; it costs nothing per invocation and is self-consistent, though determinism buys repeatability, not correctness: a gate passes any artifact wrong in a way its predicate does not express (§3.5). A *judge* is a model invocation posed one question that no gate can answer; its authority derives from the controller that enforces its verdict, not from any assumed reliability of the model (§6.4). A *guard* is loop control rather than verification: it decides whether an action is re-executed, whether unparseable output ends the run, and how much of an observation enters the transcript, but it evaluates nothing for correctness. The third term matters because guards and gates are easily conflated and, in the pilot, confounded (§5.2).

1.2 Objectives

We pursue two objectives of different scope, stated separately because the evidence for each differs.

O1, near-term: a minimal verification layer is a cheap and near-universal improvement to ordinary agents on ordinary tasks. A small set of deterministic checks—costing no model calls and expressible in a few hundred lines of ordinary code—changes the behavior of a conventional ReAct agent out of proportion to its cost: primarily by making the agent *stop for a stated reason*, and secondarily by making it more accurate. The claim's scope is deliberately modest—short tasks, one model tier, one task family—and if it holds, the recommendation is unconditional, because the layer is free at run time and does not depend on model capability.

O2, long-term: verification should be a standard architectural commitment for agent systems. In software engineering, a test layer is not an optimization adopted when time permits; it is structural, and its absence is a defect in the process. We argue that agent systems are in that state now. Concretely, an agent built to this proposal returns exactly one of two things: an answer that named verifiers accepted, or an abstention naming the level that refused and the evidence it saw—never an answer no component but its author has inspected, and never a stop forced by a token limit. The V-model pairing is what provides these guarantees: specification levels, a verifier paired with each level, deterministic control of every transition, and memory written only by verdicts. The target regime is the one in which agents are simultaneously least reliable and most in demand: long-horizon tasks with many interdependent components, where one unchecked intermediate result contaminates everything downstream. We do not claim that this paper establishes O2. We claim that O2 is well posed, that it is a candidate for a standard rather than one more technique, and that this paper provides its first measurement.

Three observations make O2 plausible rather than merely appealing. First, most of the layer is *free*: the deterministic half produced eight of the nine corrections in our study at zero marginal cost. Second, the layer does *not require a stronger model*: the gates worked at the 8B scale, at which the executor itself barely worked. Third, the structure is already validated in the discipline it is borrowed from, and the evidence against self-correction [14, 15] argues *for* externalizing the check rather than against checking. The counterweight is our own planning result (§5.3): adding a decomposition stage without the verifier meant to check it made the system markedly worse, so the structure may be adoptable only as a whole—a cost the plan in §8 must assess.

1.3 The pilot, and its scope

The pilot study implements the outer (L1/R3) and inner (L3/R1) pairs of the architecture and compares five configurations that differ by one component at a time, so that every correction can be attributed to the component that caused it. Its research questions test O1 directly and bound O2 from below. **RQ1**: What does a deterministic verification layer do to task success and to termination behavior? **RQ2**: What does an LLM judge add beyond the gates? **RQ3**: Does planning still help once verification is present? **RQ4**: What does each component cost?

We designed the study around *silent errors*: answers that are wrong but delivered without any indication that the chain failed. At the model scale we study, that failure mode did not occur—the unverified configurations never advanced far enough to be confidently wrong, and instead collapsed structurally. The contribution

therefore shifted to *termination behavior*: whether an agent halts for a stated reason or continues until refused externally, a property that matters comparably for deployment. The silent-error hypothesis is reported as untested at this scale, with qualitative observations at larger scale (§6.2). The evidence base is 47 executions over five items with one backbone, so all quantitative statements are directional; the middle pair (§3.4) and the memory schema (§3.3) are specified but not built.

In summary, this paper makes the following contributions:

- an architecture that treats verification as a layer of an agent system, pairing each specification level with a verifier under a deterministic controller (§3);
- a definition of verification for agents (V1–V4) separating gates, judges, and guards, which makes each half of the layer separately measurable (§1.1);
- a pilot study of the deterministic core: five configurations, 47 executions, every correction attributed to its component (§5);
- a twelve-month plan that supplies what the pilot lacks and moves to a code domain where the integration pair can be exercised (§8).

Workshop positioning, and the feedback we seek. This paper is a pilot, submitted to JAWs for feedback on experimental design before the larger study is committed to, and we would value disagreement on three points: (i) gates and guards are confounded (§5.2)—is separating them the right next controlled condition, or should scaling across model tiers come first? (ii) the judges are the same model that produced the work, and two repairs are available (§6.4)—a stronger judge above a weaker executor, or a judge restricted to questions the deterministic layer can ground: which makes the better second study? (iii) the integration verifier needs a task family with assembled artifacts, for which we propose SWE-bench—would a smaller domain with cheaper ground truth serve better?

2 Background

Verification and validation are distinct activities with distinct artifacts [41]. In the V-model, each design stage is paired with a test stage of matching scope: unit tests examine the detailed design, integration tests examine the architecture, and acceptance tests examine the requirements. Falcão et al. [43] report nearly two decades of its use as a framing device whose stage transitions act as quality gates, and Wu [44] finds, from practitioner interviews on ML-enabled systems, that layered decomposition, explicit responsibility boundaries, and consistent verification across levels correspond to those systems' failure modes. Our proposal rests on the observation that an agent's control loop is a delivery process of this kind—run at inference time and at high frequency—but built without the corresponding certification structure.

The failure mode that model-based verification targets, a fluent, confident, and unsupported answer, is well characterized [18]; detection methods range from sampling-based consistency [19] to attribution against context spans [20], and our deterministic grounding check is the simplest member of that family, testing whether the answer occurs in any returned observation. For the task domain, MuSiQue [24] was constructed so that single-hop shortcuts fail, which is why we adopt it over HotpotQA [23]; agentic approaches to multi-hop retrieval include Self-Ask [26] and IRCoT [25]. Broader agent benchmarks [30, 31] report that long-horizon tasks fail for structural reasons and that single-attempt accuracy overstates deployed reliability.

3 The Proposed Architecture: ReAct under the V-Model

3.1 Levels and their paired verifiers

Figure 1 shows the motivating frame and Figure 2 the architecture it yields: specification levels descend on the left, verifiers ascend on the right, and each verifier is paired with exactly one level, so the level at which a failure is caught identifies the level at which it was introduced.

L1, requirements: the request plus a machine-usable statement of what would satisfy it, such as the expected answer type or, in code, the failing test that must pass. *L2, decomposition*: the subtasks, their order, the data flow between them, and each one's completion condition; this is where most long-horizon failures are introduced, since a decomposition that mis-binds one subtask to another cannot be repaired downstream. *L3, step specification*: one tool, one argument, one expected result type; the only level the executor sees in full.

R1, unit inspects one observation against the L3 step that requested it. *R2, integration* inspects the assembled intermediate results against L2: do references resolve, do types compose, do the parts concern the entities the decomposition assumed. *R3, acceptance* inspects the final answer against L1.

3.2 Runtime properties

Three properties of the architecture are load-bearing. First, *the controller is the only stateful component*: it holds the current level, the per-level budget, and the artifact store, and the generative components are called as functions that return content. Second, *every verifier is split*: the gate half runs first and costs nothing, and the judge half runs only on artifacts the gates have cleared, and only at levels whose failure class is semantic; this split is what makes the two halves separately priceable. Third, *rejection routes to the paired level*: R1 re-issues one step; R2 returns to L2 and re-decomposes, discarding only what is downstream of the mis-binding; R3 returns to L1 or, once the budget is exhausted, abstains. The system's output is therefore a triple: an answer or an abstention, the level of the last rejection, and the evidence inspected.

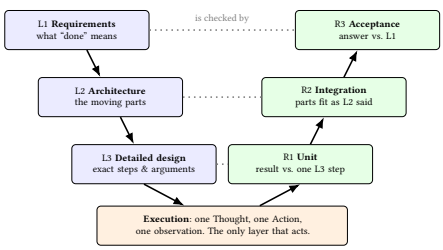


Figure 1: The motivating frame. Solid arrows show the flow of work; dotted lines show which verifier checks which specification. The pilot implements the outer pair (L1/R3) and the inner pair (L3/R1) only.

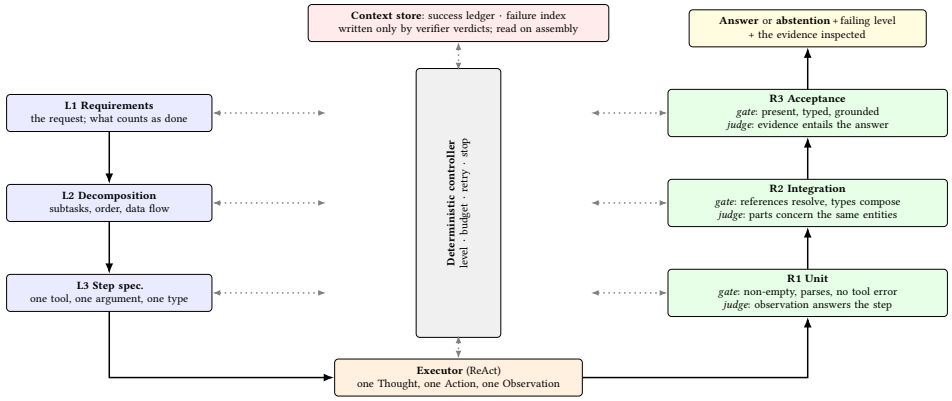


Figure 2: The architecture. Dotted lines are the controller's mediation: no artifact reaches a verifier and no verdict reaches a generator except through it, and it alone reads and writes the context store (§3.3). Each verifier is a free deterministic gate followed by an optional LLM judge invoked only on what the gate cleared; a rejection routes to its paired level, and an exhausted budget yields an abstention naming that level. The pilot implements the R3 and R1 rows only.

One consequence holds by construction rather than by model behavior. *Bounded termination with cause*: budgets are finite constants and every transition is decided by finite program logic, so every run terminates within bounded steps in one of exactly two states—an accepted answer or an abstention naming the failing level—under no assumption about the backbone; the uncapped collapse of §4 is what its absence looks like.

3.3 Memory: what each component may remember

In a ReAct loop the transcript *is* the memory: every thought, action, observation, and failed attempt is resubmitted on each turn, which is why token consumption grows quadratically [7] and why, late in a task, most of what the model reads concerns steps it has already finished or already got wrong. Nothing in the loop decides what belongs there. Our design replaces accumulation with assembly: the controller owns two stores, a verdict is the only event that writes to either, and each invocation receives a context built to a declared schema (Table 2, Figure 3).

The *success ledger* is append-only and written only when a verifier accepts: the subtask, the condition it satisfied, the accepted output or interface, and a pointer to the evidence; rejected attempts and raw transcripts never enter it, so it is by construction the set of steps that moved the task forward. The *failure index* is keyed by component and condition, recording which check rejected, why, and what the retry did; a retry receives only the entries matching its own component and condition, which turns a pile of failed attempts into a line reading *rejected twice by the type gate for the same missing field* and makes repetition a stopping condition the controller can read. *Isolation* makes both stores usable: the executor is invoked per subtask, and a subtask is a new input rather than a continuation, receiving its objective, contract, and checks—and none of the requirements, architecture, sibling implementations, or earlier transcripts.

Two consequences follow, both design expectations rather than results: the executor's context is bounded by the largest subtask instead of by trajectory length, and abstention becomes *scoped*, since work banked in the ledger survives a component's failure. Neither is implemented in the pilot, which keeps a conventional transcript with a truncation guard; §8 schedules the schema as an ablation.

Table 2: The context schema. The executor row is what makes a subtask a new input rather than a continuation.

Role	Reads	Never reads
Planner (L2)	L1; inventory of tools and components	any execution output
Designer (L3)	L2; this component's slot and contract	sibling implementations
Executor (one subtask)	**its objective; the contract it must honor; its checks; its own failure entries**	**L1, L2, sibling code, every earlier transcript**
Gates	the artifact and the one condition	everything else
Judges R1–R3	the artifact and its paired specification level	every other level, and all transcripts
Controller	the whole state	—

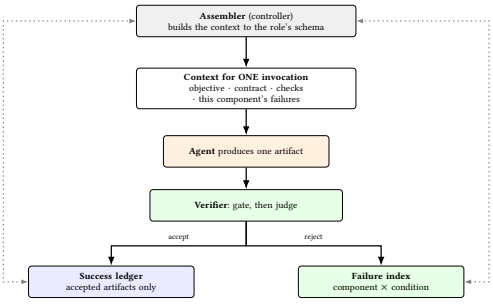


Figure 3: The write path. A verdict is the only event that writes memory, so a rejected attempt survives as its verdict and not as a transcript, and the assembler reads back only what Table 2 permits.

3.4 What is new, and what the pilot omits

Each ingredient of the architecture has precedent; the arrangement does not. Deterministic control of traversal appears in EoG [22], external per-step signals appear in process supervision [16] and interpreter feedback [17], and model-based critique appears in Reflexion and Self-Refine [11, 12]. What is absent from all of them is a verification subsystem that is *stratified by specification level*, so that rejections localize; *external and binding* by construction, so that the checked party cannot overrule the check; *split into free and paid halves at every level*, so that the cost of judging becomes

an empirical question; and coupled to a *memory written only by verdicts*. O2 is the claim that this arrangement, not any one part, is what turns an agent into a component other software can call.

The pilot omits the middle (L2/R2) pair because multi-hop question answering assembles nothing: every subtask is a retrieval returning a string, so an integration verifier has nothing to inspect beyond what L3/R1 already compares. This is a property of the task family, not of the architecture, and it is why §8 moves to code, where assembled artifacts and executable ground truth both exist. §3.5 presents a pilot failure that only an integration verifier could have caught.

3.5 Worked example: a four-hop question

The item asks for the son of the Italian navigator who explored, for England, the eastern coast of the continent on which Ignacio Esparza's birthplace lies; the gold answer is *Sebastian Cabot*. A VPG run produced this accepted plan (L2), with what each subtask returned:

1. Find the region where Ignacio Esparza was born.
   → *Guadalajara, Jalisco, Mexico* (correct)
2. Find the navigator father of the person whose name is associated with the region in the result of subtask 1.
   → *Giovanni Battista Caviglia* (wrong)
3. Find the explorer who explored the east coast of the region in the result of subtask 2.
   → *Lieutenant James Cook* (wrong, inherited)
4. Find the navigator whose name is the result of subtask 3.
   → *no answer; retrieval continues until the budget ends*

Every R1 check passes at every step: each subtask returned a non-empty, correctly typed result, free of tool errors and present in a retrieved paragraph, and R3 never fires because no final answer is produced. The fault lies at L2 and is invisible to both implemented pairs: subtask 3 is defined over *the result of subtask 2* rather than over the question, so the distractor matched at step 2 propagates. This is an integration failure in the V-model sense—individually valid units assembled into an invalid whole. An R2 gate comparing the type subtask 2 produced (a *person*) with the type subtask 3 presupposes (a *region*) would reject the chain at step 3 with no model call and return control to L2 for re-decomposition; the judge half would handle the residual case in which types agree but entities do not. Whether type-composition gates fire often enough to pay for themselves is an empirical question the plan places first among the code-domain experiments.

3.6 Worked example: fixing a repository issue

A SWE-bench task supplies a repository, an issue, a test the patch must make pass, and a suite that must keep passing [32]—two test sets at different specification levels, precisely the structure the architecture exploits. Consider an issue reporting that timestamps lose their timezone offset:

- *L1* the issue plus its failing test.
- *L2* two files change, the serializer and the parser, and their contract is that a timestamp carries an offset.
- *L3* one edit at a time, each with the tests that cover it. The executor is given *one edit, its contract and its tests*—not the issue, not the repository, not the previous edit's transcript.
- *R1* the patch applies, the file parses, its own tests pass. Free, and no model involved.
- *R2* the rest of the suite. The fix passes its own test and breaks three that parsed the old format: every edit is correct alone, the assembly is not. Control returns to L2, not to the executor, and only the edits downstream of the revised contract are redone.
- *R3* the original failing test, plus the one question no gate can answer: does the patch fix the issue, or only the test that came with it?

If an edit is rejected three times by the same gate, the run stops with *failed at R1, edit 2, patch does not apply*, keeping what was already accepted. A monolithic agent emits patches until its budget ends and returns a transcript in which nothing separates the useful attempt from the discarded ones.

3.7 The pilot implementation

The implemented pipeline (Figure 4) is Figure 2 without the R2 row and without the context store. Three principles govern the implementation. First, control resides in program logic: steps, retries, and termination are decided in ordinary code bounded by named constants. Second, inexpensive checks precede expensive ones, so that a judge is paid for only where gates cannot decide. Third, failures are reported rather than reinterpreted, because silently reformulating a step into a weaker objective is the mechanism by which agents come to fabricate values.

Eleven gates examine the plan: nine structural checks (parseability, length, duplicate subtasks, self-, forward and dangling references, terminal step, typed completion conditions, tool existence) and two that apply only to arithmetic questions (single-expression form and numeric coverage). Seven examine an answer (presence, non-emptiness, non-answers, consistency with the alternatives a comparison offers, tool errors, type agreement, and optionally occurrence in an observation). The plan judge asks whether the steps, assuming each succeeds, would answer the question; the answer judge asks whether the evidence entails the answer. Three guards operate inside the executor—a repeated action is served from cache with a warning, unparseable output ends the run after bounded attempts, and observations are truncated before entering the transcript; per §1.1 these are loop control, not verification.

Table 3: The five configurations; internal labels in parentheses match the released logs. All share one executor, tool set, retriever, scorer and backbone, so adjacent pairs differ by one component.

Config.	Composition	Plan	Guards	Gates	Judges
N (A)	plain ReAct	–	–	–	–
NP (B)	ReAct + planner	yes	–	–	–
VG (C1)	verified, no planner	–	yes	yes	–
VPG (C0)	+ planner + plan gates	yes	yes	yes	–
VPGJ (C)	+ LLM judges	yes	yes	yes	yes

Table 3 defines the configurations. VG proves central to the analysis: it has no planner—the question itself is the single task, carrying a completion condition derived from its interrogative form, so that "Who …" yields *a person or organization, not one of their attributes*. VG is the minimal configuration that retains verification.

4 Experimental Design

Establishing O2 requires five methodological ingredients, most of which the pilot lacks by design: (i) *assembled artifacts*, without

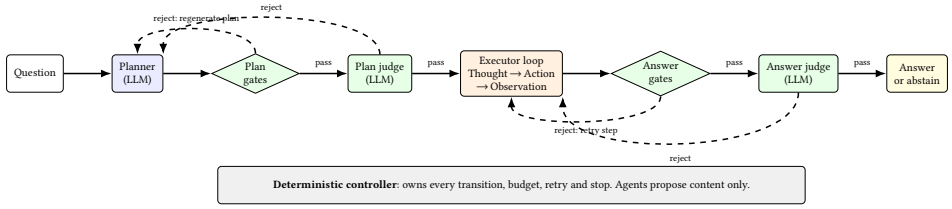


Figure 4: The implemented pipeline. Diamonds are deterministic checks (no model cost); green rounded boxes are LLM judges, invoked only on gate-cleared artifacts; dashed arrows are bounded rejection paths. When plan regeneration is exhausted, the system falls back to executing the question as a single task (§5).

which R2 has nothing to inspect; (ii) *executable ground truth*, without which a verifier's mistakes cannot be distinguished from the scorer's; (iii) *several model tiers*, because each half of the layer addresses a failure class that appears at a different scale; (iv) *abstention-aware measures*, because accuracy alone cannot summarize a system that is permitted to decline; and (v) a *switchable memory schema*, without which context management and verification are measured together. The plan in §8 is organized around supplying all five. The pilot has the fourth and part of the third: it is a feasibility study, showing that architectural differences are measurable at all when every other component is held constant.

Task, tools, model. MuSiQue-Answerable [24], four-hop stratum: twenty paragraphs per item, of which roughly four support the answer and the rest are mined distractors. We use five items, one of which also appears in a four-paragraph gold-context variant as a control of reduced difficulty. Two tools are bound to a single item's paragraphs: `retrieve[query]`, a BM25 [28] index returning three query-centred excerpts with indices, and `read_paragraph[idx]`. A single allow-list function is the only path from harness to tool, emitting nothing beyond index, title, and text, so the gold answer and the supporting-paragraph annotations cannot reach the retriever. Every role in every configuration runs `llama-3.1-8b-instant` at temperature 0, so differences are attributable to architecture and not capacity; the executor budget is 12 steps unless stated otherwise, over two passes.

Measures. A verified configuration may decline to answer and an unverified one cannot, which a single accuracy figure cannot represent. We therefore report a partition summing to unity: *correct*; *silent error* (an answer was produced and is wrong—the consequential case, since nothing downstream can distinguish it from a correct answer); and *abstention*; plus *coverage*, the proportion of scoreable runs that answered. Scoring follows MuSiQue's SQuAD-style procedure [29]. A *correction* is a rejection whose retry produced a *different* answer that was then accepted; the change requirement matters, since accepting a resubmitted identical answer is the judge contradicting itself.

Execution accounting. The study comprises *47 executions*: five items over two passes for four configurations, plus seven for VPGJ, whose second pass was replaced by two runs with the plan judge disabled to observe an executed plan. Of these, 37 were scoreable and 10 were not, a distribution that is itself a result (Table 4). Three points follow. First, the second pass for N and NP ran with the step cap removed, and *all ten of those runs terminated without scoreable output*, each invoking the same tool until the provider's token limit ended the process. Second, repeated passes for VG and VPG reproduced the first pass exactly—decoding determinism at temperature 0, not stability—so the per-run costs in Table 5 are computed over distinct logged traces. Third, the plan judge in VPGJ rejected every plan in all five of its enabled runs, and the two runs with the judge disabled both returned no answer; this is the smallest cell in the study and is reported separately.

5 Results of the Pilot

5.1 Execution outcomes (RQ1)

Table 4: Execution accounting over all 47 runs. No configuration produced a silent error, so that column is omitted.

Config.	exec.	score.	crash	corr.	abst.	cov.
N (plain ReAct)	10	5	**5**	**0**	5	0.00
NP (+ planner)	10	5	**5**	**0**	5	0.00
VG (verified)	10	10	0	**8**	2	0.80
VPG (+ planner)	10	10	0	1	9	0.10
VPGJ (+ judges)	7	7	0	**4**	3	0.57
Total	**47**	**37**	**10**	**13**	**24**	—

N and NP answered none of the ten questions: all ten scoreable runs hit the 12-step cap without once invoking `finish`, all ten uncapped runs continued until the provider refused them, and the study's entire 21% crash rate falls on these two configurations. VG answered eight of ten, using fewer steps than N (Table 5), and abstained on the two items it could not resolve instead of guessing.

5.2 A confound between guards and gates

One qualification applies to every result below. The verified configurations differ from N and NP in two respects: the gates, which are verification, and the three guards, which are loop control. The collapse mode of §6.1, in which an agent invokes the same tool until an external limit intervenes, is plausibly prevented by the duplicate-action guard alone, which refuses the second identical invocation without consulting any gate. We therefore cannot attribute the observed difference to verification as such; separating the two is the first ablation in §8.

5.3 The effect of planning (RQ3)

VPGJ with the plan judge enabled and VG produced identical outcomes: the judge rejected all five plans, so VPGJ fell back to executing the question as a single task and became VG with additional overhead. Partitioning every run by whether a plan was executed makes the effect explicit. Twelve runs executed a plan, of which one was correct (a success rate of 0.08), at a mean of 15.6 executor steps; fifteen runs executed no plan, of which twelve were correct

Table 5: Mean cost per scoreable run over distinct logged traces. Executor steps are the architecture-neutral comparison; total calls include planner and judge overhead.

Config.	LLM calls	exec. steps	prompt tok.	compl. tok.
N	12.0	12.0 (all at cap)	18,891	598
NP	13.0	12.0 (all at cap)	11,180	752
VG	**11.8**	**11.8** (3–21)	19,511	1,150
VPG	15.2	14.2 (8–20)	17,226	840
VPGJ	14.8	10.0 (4–18)	20,943	1,210
judge disabled	23.0	20.0	28,609	1,464

Table 6: Rejections and corrections across the logged traces; identical repeats are not double-counted. A correction required the retry to produce a different answer that was then accepted.

Source	rejections	corrections	rate	marginal cost
Deterministic gates	24	**8**	0.33	none
LLM judges	3	1	0.33	one call each

(0.80), at a mean of 10.9 steps. Planning thus cost 43% more steps for roughly one tenth of the success rate.

Two mechanisms appear in the traces. *Budget fragmentation*: four subtasks under a twelve-step budget allow three steps each, the minimum viable trajectory (retrieve, read, finish), leaving no capacity to reformulate a failed query, and most planned failures end at the first or second subtask. *Error propagation*: the chain fails at its first incorrect resolution, as §3.5 shows in full, and no gate in the implemented system can detect it, since each subtask answer is present, non-empty, typed and grounded and only the composition is wrong; no configuration solved that item in any run. This is the missing L2/R2 pair appearing as a measured result rather than an argument.

5.4 Cost (RQ4)

The distribution matters more than the mean: N and NP sat at the cap in every scoreable run, having no way to stop earlier, whereas VG ranged from 3 to 21 steps—resolving one item in three and spending its full allocation plus a retry on the item it declined—the observable signature of adaptive termination.

5.5 Attribution of corrections (RQ2)

The two halves of the layer show a comparable success rate per rejection but differ by an order of magnitude in volume: gates fired 24 times, judges three times, with no judge reversal (rejecting and then accepting the identical answer). The plan judge behaved differently from the answer judge: it rejected all five plans with a recurring objection of the form *"the plan does not guarantee that X is the same as Y"*, twice identically across both regeneration attempts. No plan can guarantee what a retrieval will return, so regeneration cannot satisfy such an objection, and the fallback to unplanned execution becomes deterministic.

5.6 Retrieval ceiling

The agent reached 67% of the required paragraphs on average, and all of them in 25% of runs, which bounds every configuration: architectural differences are measured on a retriever that omits about a third of the necessary evidence, because the executor issues sentence-form queries against a lexical index. VG answered eight of ten while reaching complete evidence in only one run of five, so several items were resolved from a subset of the annotated evidence.

This ceiling bounds verifier calibration in both directions. The judge never sees the gold answer or the supporting-paragraph annotations, so when required evidence is never fetched it can reject an answer the scorer would have accepted, inflating abstentions. We observed no such false negative: the three judge rejections concerned intermediate results the scorer does not evaluate, and every final-answer abstention came from a gate observing that no answer existed. We observed no false positives either, but attach little weight to that, since no configuration produced a confidently wrong answer for a judge to accept.

6 Discussion

6.1 Unverified agents do not abstain

No unverified execution ended with a refusal: the recorded abstentions of N and NP in Table 4 are the harness observing that the loop ended without invoking `finish` once the step cap intervened, and removing the cap—our artifact, not ReAct's—produced the uncapped collapse of §4. The verified configurations differ in kind: every termination is a decision with a recorded cause. Independently of accuracy, halting with a stated reason is what lets a component be composed into a larger system; we therefore regard the deterministic half as close to a precondition for unattended deployment, subject to the confound of §5.2.

6.2 Behavior under a stronger backbone

No confidently wrong answers occurred, so the silent-error hypothesis could not be tested; we report this as a null result. Using the model on which the judge contributes least was deliberate: *the 8B experiment establishes a lower bound*. When reasoning capacity is identical and low across configurations, whatever separates them is attributable to guards, gates, and control flow; at 70B the backbone absorbs the failures the architecture exists to handle, measuring the model instead of the design. The cost is that the judge's target failure class does not arise.

Exploratory runs on two larger backbones (`llama-3.3-70b-versatile` and `qwen/qwen3.6-27b`), outside the controlled set and requiring confirmation, indicate where that failure class does arise. With a stronger backbone, N and NP stop collapsing structurally: they hold the action format, terminate, and produce answers. On harder items, however, the loop commits to a plausible intermediate entity and returns a fluent answer with no indication that the chain broke. A recurring proportion of these answers were rejected by the answer judge and by no gate, for reasons citing exactly what gates cannot evaluate: the evidence does not support the answer, the observation concerns a different entity, or a plausible name was inferred rather than retrieved—the intrinsically semantic class the hallucination literature [18, 19] targets. The implication is a division of labor: gates are mandatory, being free and covering the malformed, empty, mistyped, repeated, and placeholder outputs that dominate at every scale we examined; judging should be deployed selectively, where unsupported but well-formed output is the anticipated failure.

6.3 Planning and verification are partial substitutes

Also outside the controlled set, on stronger backbones NP consistently outperformed N, in line with ReWOO and ADaPT [7, 9]; that advantage did not survive verification (8 of 10 versus 1 of 10). A tentative explanation is that a planner's benefit to an unverified loop is structural discipline, which guards and gates supply more cheaply; what remains is the decomposition itself, which on retrieval-shaped tasks fragments the budget and propagates the first error. The architectural reading is that a decomposition is an L2 artifact and the pilot has no R2 verifier to check one; degradation from adding L2 without R2 is what the architecture predicts. Whether planning recovers its value once paired with integration verification is a question for §8.

6.4 Why self-assessment is structurally limited

We attribute the judge's weakness at 8B to its configuration, not to prompt quality; the prompts were revised repeatedly, and the final failures used the revised rubric's own vocabulary. The judges are the same model at the same temperature, examining the same tool outputs and evaluating the same model's work, so they share the actor's blind spots and are least reliable exactly where the actor is. Their available behaviors reduce to agreement, which conveys nothing, and unanchored disagreement, which is noise. This is the configuration Huang et al. [14] report to fail, and it violates V1 in spirit while satisfying it in letter.

Two designs would constitute a fair test of model-based judging. *Asymmetric verification* places a strictly stronger model in the judge roles above a weaker executor; judges issue one to two calls per subtask against the executor's seven to thirteen, so the cost is modest (§6.2 provides indirect evidence in its favor). *Ground-truth-constrained verification* allows the judge only questions whose evidence the deterministic layer supplies (e.g., whether a specified span entails the answer [16, 17]); its limit is that it cannot detect a failure the deterministic layer did not already suspect.

Finally, what the judge did catch—an executor emitting the literal placeholder `finish[...]`, admitted by all seven gates—was syntactic, and expressing it as a gate afterwards took three lines of code. This suggests a practice: run a judge during construction, promote each recurring catch into a gate, and reserve run-time judging for the semantic class above, which cannot be promoted.

7 Threats to Validity

We follow the threat categories conventional in empirical software engineering [39, 40].

Internal validity. The five configurations are implemented as separate directories; the executor, gates, tools, and scorer are byte-identical across them and every run's configuration is logged, but divergence remains a risk we detected and corrected more than once during development.

Construct validity. Exact-match scoring under-credits paraphrases, and the retrieval ceiling (§5.6) bounds every reported figure and may generate false negatives for the judge.

External validity. The study uses one backbone, one benchmark, one task family, and the small sample of §4; §6.2 reports qualitative patterns, not measurements; and the architecture is evaluated only in part, the integration row and the memory schema being argued for rather than tested.

Conclusion validity. With 37 scoreable runs and cells of 7 to 10, no statistical inference is performed and none should be drawn; the absent runs are not missing at random but are the crashes of Table 4, so N and NP appear in the outcome columns only in their capped configuration.

Researcher bias. One author implemented all five configurations, and scoring was not blind.

8 One-Year Research Plan

Four deficiencies drive the plan: insufficient statistical power, a single model tier, a task family that cannot exercise the integration layer, and separate per-configuration directories. Table 7 organizes the schedule around them.

Table 7: Twelve-month plan; half the schedule goes to code, where deterministic verifiers are rich and the L2/R2 row can be exercised.

Months	Work and rationale
1–2	**Consolidation, and the confound.** One codebase with the configuration as a parameter, removing the divergence risk of §7; a dense retriever [27] to raise the 67% ceiling. *First ablation:* guards without gates, gates without guards, and both ends of the ladder—until those cells exist, the difference in §5 cannot be assigned to verification rather than loop control.
3–4	**Scale, and verifier configurations.** Roughly 500 items stratified by hop count, three seeds, three to five model tiers, every repeat retained; paired per-item analysis with effect sizes [38], hypotheses registered beforehand. Asymmetric and ground-truth-constrained verification (§6.4) as separate conditions. *Second ablation:* full accumulation vs. ledger only vs. both stores, with tokens read per step reported alongside accuracy.
5–7	**Transfer to code.** Port to SWE-bench [32] and its agent interfaces [33], with function-level suites under rigorous test augmentation [34, 35] as a cheaper inner loop. In code the gate layer becomes a compiler, type checker, and test runner, with ground truth no question-answering gate can have; we expect the gate/judge balance to shift and will re-run Table 6.
8–10	**Debugging, and the integration layer.** Fault localization and repair, where a failing test supplies both trigger and oracle [17]. The L2/R2 pair becomes testable at last: individually correct units forming a broken assembly is a real repository failure class invisible to unit checks (§3.5). Repository length makes the memory schema consequential, so its ablation is repeated with per-component isolation on and off.
11–12	**Synthesis.** Cross-domain comparison of where each half contributes, a replication package of code, prompts, items, seeds and logs, and submission.

9 Conclusion

This paper argued that LLM agents run a delivery process without the certification structure such a process needs, transferred the V-model to supply that structure, and measured its deterministic core. The measurement is small—five items, 47 executions, one 8B backbone—but its central contrast is stark: without the layer, no run ever terminated by decision; with it, every run ended in an accepted answer or a reasoned abstention, and eight of nine corrections were free. Planning, added without its paired verifier, made the system worse—the architecture's own prediction, appearing as data.

Three uncertainties bound what this establishes, and they are the feedback we seek at the workshop: whether the effect belongs to verification or to the bundled guards (§5.2); whether judges recover their value once separated from the model they check (§6.4); and whether the integration level, which no question-answering task can exercise, earns its place once the architecture moves to code (§8). What we ask the field to weigh, meanwhile, is not another technique for a leaderboard, but whether an agent without an external verification layer is structurally incomplete, in the way untested software is.

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023. arXiv:2210.03629.

[2] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," in *NeurIPS*, 2022.

[3] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," in *Proc. ICLR*, 2023. arXiv:2203.11171.

[4] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of Thoughts: Deliberate Problem Solving with Large Language Models," in *NeurIPS*, 2023. arXiv:2305.10601.

[5] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language Models Can Teach Themselves to Use Tools," in *NeurIPS*, 2023. arXiv:2302.04761.

[6] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.

[7] B. Xu, Z. Peng, B. Lei, S. Mukherjee, Y. Liu, and D. Xu, "ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models," arXiv:2305.18323, 2023.

[8] Molinari and Ciravegna, "Reason-Plan-ReAct: A Reasoner-Planner Supervising a ReAct Executor for Complex Enterprise Tasks," arXiv:2512.03560, 2025.

[9] A. Prasad, A. Koller, M. Hartmann, P. Clark, A. Sabharwal, M. Bansal, and T. Khot, "ADaPT: As-Needed Decomposition and Planning with Language Models," arXiv:2311.05772, 2023.

[10] "Why Reasoning Fails to Plan: A Planning-Centric Analysis of Long-Horizon Decision Making in LLM Agents," arXiv:2601.22311, 2026.

[11] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language Agents with Verbal Reinforcement Learning," in *NeurIPS*, 2023. arXiv:2303.11366.

[12] A. Madaan et al., "Self-Refine: Iterative Refinement with Self-Feedback," in *NeurIPS*, 2023. arXiv:2303.17651.

[13] Y. Bai et al., "Constitutional AI: Harmlessness from AI Feedback," arXiv:2212.08073, 2022.

[14] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large Language Models Cannot Self-Correct Reasoning Yet," in *Proc. ICLR*, 2024. arXiv:2310.01798.

[15] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *NeurIPS Datasets and Benchmarks*, 2023. arXiv:2306.05685.

[16] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's Verify Step by Step," in *Proc. ICLR*, 2024. arXiv:2305.20050.

[17] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching Large Language Models to Self-Debug," in *Proc. ICLR*, 2024. arXiv:2304.05128.

[18] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[19] P. Manakul, A. Liusie, and M. J. F. Gales, "SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models," in *Proc. EMNLP*, 2023. arXiv:2303.08896.

[20] Wang et al., "AttnTrace: Contextual Attribution of Prompt Injection and Knowledge Corruption," arXiv:2508.03793, 2025.

[21] Liu et al., "QuadSentinel: Sequent Safety for Machine Learning Agents," arXiv:2512.16279, 2025.

[22] Jha et al., "Think Locally, Explain Globally: Graph-Guided LLM Investigations via Local Reasoning and Belief Propagation," arXiv:2601.17915, 2026.

[23] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," in *Proc. EMNLP*, 2018.

[24] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop Questions via Single-hop Question Composition," *TACL*, vol. 10, pp. 539–554, 2022.

[25] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions," in *Proc. ACL*, 2023. arXiv:2212.10509.

[26] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and Narrowing the Compositionality Gap in Language Models," in *Findings of EMNLP*, 2023. arXiv:2210.03350.

[27] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proc. EMNLP*, 2020. arXiv:2004.04906.

[28] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[29] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ Questions for Machine Comprehension of Text," in *Proc. EMNLP*, 2016.

[30] X. Liu et al., "AgentBench: Evaluating LLMs as Agents," in *Proc. ICLR*, 2024. arXiv:2308.03688.

[31] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains," arXiv:2406.12045, 2024.

[32] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *Proc. ICLR*, 2024. arXiv:2310.06770.

[33] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," in *NeurIPS*, 2024. arXiv:2405.15793.

[34] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.

[35] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation," in *NeurIPS*, 2023. arXiv:2305.01210.

[36] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Trans. Softw. Eng. Methodol.*, 2024. arXiv:2308.10620.

[37] A. Fan, B. Gokkaya, M. Harman, M. Lyu, J. Mao, M. Mytkowicz, and S. Yoo, "Large Language Models for Software Engineering: Survey and Open Problems," in *Proc. ICSE-FoSE*, 2023. arXiv:2310.03533.

[38] R. Dror, G. Baumer, S. Shlomov, and R. Reichart, "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing," in *Proc. ACL*, 2018, pp. 1383–1392.

[39] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Springer, 2012.

[40] P. Runeson and M. Höst, "Guidelines for Conducting and Reporting Case Study Research in Software Engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

[41] B. W. Boehm, "Guidelines for Verifying and Validating Software Requirements and Design Specifications," in *Proc. Euro IFIP*, 1979, pp. 711–719.

[42] M. Leucker and C. Schallhart, "A Brief Account of Runtime Verification," *Journal of Logic and Algebraic Programming*, vol. 78, no. 5, pp. 293–303, 2009.

[43] R. Falcão, A. Jedlitschka, F. Elberzhager, and D. Rombach, "Experiences in Using the V-Model as a Framework for Applied Doctoral Research," arXiv:2407.04563, 2024.

[44] J. JW Wu, "An Exploratory Study of V-Model in Building ML-Enabled Software: A Systems Engineering Perspective," in *Proc. CAIN*, Lisbon, Portugal, 2024, pp. 1–11. arXiv:2308.05381.